

Predicting Stock (Dow Jones Industrial Average (^DJII)) Prices with Artificial Neural Networks

Building neural networks

This practical assignment will start with implementing a shallow network from scratch, followed by a deep network with two layers using scikit-learn. We will then implement a deep network with TensorFlow and Keras.

1) Implementing neural networks from scratch

We will use sigmoid as the activation function in this example. We first define the sigmoid function and its derivative function:

```
>>> def sigmoid(z):  
...     return 1.0 / (1 + np.exp(-z))  
>>> def sigmoid_derivative(z):  
...     return sigmoid(z) * (1.0 - sigmoid(z))
```

You can derive the derivative yourself if you are interested in verifying it. We then define the training function, which takes in the training dataset, the number of units in the hidden layer (we will only use one hidden layer as an example), and the number of iterations:

```
>>> def train(X, y, n_hidden, learning_rate, n_iter):  
...     m, n_input = X.shape  
...     W1 = np.random.randn(n_input, n_hidden)  
...     b1 = np.zeros((1, n_hidden))  
...     W2 = np.random.randn(n_hidden, 1)  
...     b2 = np.zeros((1, 1))  
...     for i in range(1, n_iter+1):  
...         Z2 = np.matmul(X, W1) + b1  
...         A2 = sigmoid(Z2)  
...         Z3 = np.matmul(A2, W2) + b2  
...         A3 = Z3  
...         dZ3 = A3 - y  
...         dW2 = np.matmul(A2.T, dZ3)  
...         db2 = np.sum(dZ3, axis=0, keepdims=True)  
...         dZ2 = np.matmul(dZ3, W2.T) * sigmoid_derivative(Z2)  
...         dW1 = np.matmul(X.T, dZ2)  
...         db1 = np.sum(dZ2, axis=0)  
...         W2 = W2 - learning_rate * dW2 / m  
...         b2 = b2 - learning_rate * db2 / m  
...         W1 = W1 - learning_rate * dW1 / m  
...         b1 = b1 - learning_rate * db1 / m  
...         if i % 100 == 0:  
...             cost = np.mean((y - A3) ** 2)  
...             print('Iteration %i, training loss: %f' %  
...                   (i, cost))  
...     model = {'W1': W1, 'b1': b1, 'W2': W2, 'b2': b2}  
...     return model
```

Note that besides weights, W , we also employ bias, b . Before training, we first randomly initialize weights and biases. In each iteration, we feed all layers of the network with the latest weights and biases, then calculate the gradients using the

backpropagation algorithm, and finally update the weights and biases with the resulting gradients. For training performance inspection, we print out the loss and the MSE for every 100 iterations.

To test the model, we will use **Boston** house prices as the toy dataset ([link](#)). As a reminder, data normalization is usually recommended whenever gradient descent is used. Hence, we will standardize the input data by removing the mean and scaling to unit variance:

```
>>> boston = datasets.load_boston()
>>> num_test = 10 # the last 10 samples as testing set
>>> from sklearn import preprocessing
>>> scaler = preprocessing.StandardScaler()
>>> X_train = boston.data[:-num_test, :]
>>> X_train = scaler.fit_transform(X_train)
>>> y_train = boston.target[:-num_test].reshape(-1, 1)
>>> X_test = boston.data[-num_test:, :]
>>> X_test = scaler.transform(X_test)
>>> y_test = boston.target[-num_test:]
```

With the scaled dataset, we can now train a one-layer neural network with 20 hidden units, a 0.1 learning rate, and 2000 iterations:

```
>>> n_hidden = 20
>>> learning_rate = 0.1
>>> n_iter = 2000
>>> model = train(X_train, y_train, n_hidden, learning_rate, n_iter)
Iteration 100, training loss: 13.500649
Iteration 200, training loss: 9.721267
Iteration 300, training loss: 8.309366
Iteration 400, training loss: 7.417523
Iteration 500, training loss: 6.720618
Iteration 600, training loss: 6.172355
Iteration 700, training loss: 5.748484
Iteration 800, training loss: 5.397459
Iteration 900, training loss: 5.069072
Iteration 1000, training loss: 4.787303
Iteration 1100, training loss: 4.544623
Iteration 1200, training loss: 4.330923
Iteration 1300, training loss: 4.141120
Iteration 1400, training loss: 3.970357
Iteration 1500, training loss: 3.814482
Iteration 1600, training loss: 3.673037
Iteration 1700, training loss: 3.547397
Iteration 1800, training loss: 3.437391
Iteration 1900, training loss: 3.341110
Iteration 2000, training loss: 3.255750
```

Then, we define a prediction function, which will take in a model and produce the regression results:

```
>>> def predict(x, model):  
...     w1 = model['w1']  
...     b1 = model['b1']  
...     w2 = model['w2']  
...     b2 = model['b2']  
...     A2 = sigmoid(np.matmul(x, w1) + b1)  
...     A3 = np.matmul(A2, w2) + b2  
...     return A3
```

Finally, we apply the trained model on the testing set:

```
>>> predictions = predict(X_test, model)
```

Print out the predictions and their ground truths to compare them:

```
>>> print(predictions)  
[[16.28103034]  
 [19.98591039]  
 [22.17811179]  
 [19.37515137]  
 [20.5675095 ]  
 [24.90457042]  
 [22.92777643]  
 [26.03651277]  
 [25.35493394]  
 [23.38112184]]  
>>> print(y_test)  
[19.7 18.3 21.2 17.5 16.8 22.4 20.6 23.9 22. 11.9]
```

After successfully building a neural network model from scratch, we will move on to the implementation with scikit-learn.

2) Implementing neural networks with scikit-learn

We will utilize the MLPRegressor class (MLP stands for multi-layer perceptron, a nickname for neural networks):

```
>>> from sklearn.neural_network import MLPRegressor  
>>> nn_scikit = MLPRegressor(hidden_layer_sizes=(16, 8),  
...                           activation='relu', solver='adam',  
...                           learning_rate_init=0.001,  
...                           random_state=42, max_iter=2000)
```

The `hidden_layer_sizes` hyperparameter represents the number of hidden neurons. In this example, the network contains two hidden layers with 16 and 8 nodes, respectively. ReLU activation is used. We fit the neural network model on the training set and predict on the testing data:

```
>>> nn_scikit.fit(X_train, y_train)
>>> predictions = nn_scikit.predict(X_test)
>>> print(predictions)
[16.79582331 18.55538023 21.07961496 19.21362606 18.50955771 23.5608387
22.27916529 27.11909153 24.70251262 22.05522035]
```

And we calculate the MSE on the prediction:

```
>>> print(np.mean((y_test - predictions) ** 2))
13.933482332708781
```

We've implemented a neural network with scikit-learn. Let's do so with TensorFlow in the next part.

3) Implementing neural networks with TensorFlow

In the industry, neural networks are often implemented with TensorFlow. Other popular DL (multilayer neural network) frameworks include PyTorch. Now let's implement neural networks with TensorFlow by following steps:

1. First, we import the necessary modules and set a random seed, which is recommended for reproducible modeling:

```
>>> import tensorflow as tf
>>> from tensorflow import keras
>>> tf.random.set_seed(42)
```

2. Next, we create a Keras Sequential model by passing a list of layer instances to the constructor, including two fully connected hidden layers with 20 nodes and 8 nodes, respectively. And again, ReLU activation is used:

```
>>> model = keras.Sequential([
...     keras.layers.Dense(units=20, activation='relu'),
...     keras.layers.Dense(units=8, activation='relu'),
...     keras.layers.Dense(units=1)
... ])
```

3. And we compile the model by using Adam as the optimizer with a learning rate of 0.02 and MSE as the learning goal:

```
>>> model.compile(loss='mean_squared_error',
...               optimizer=tf.keras.optimizers.Adam(0.02))
```

The Adam optimizer is a replacement for the stochastic gradient descent algorithm. It updates the gradients adaptively based on training data. For more information about Adam, check out the paper at <https://arxiv.org/abs/1412.6980>.

4. After defining the model, we now train it against the training set:

```

>>> model.fit(X_train, y_train, epochs=300)
Train on 496 samples
Epoch 1/300
496/496 [=====] - 1s 2ms/sample - loss: 459.1884
Epoch 2/300
496/496 [=====] - 0s 76us/sample - loss: 102.3990
Epoch 3/300
496/496 [=====] - 0s 62us/sample - loss: 35.7367
.....
.....
Epoch 298/300
496/496 [=====] - 0s 60us/sample - loss: 2.8095
Epoch 299/300
496/496 [=====] - 0s 60us/sample - loss: 3.0976
Epoch 300/300
496/496 [=====] - 0s 56us/sample - loss: 3.3194

```

We fit the model with 300 iterations. In each iteration, the training loss (MSE) is displayed.

- Finally, we use the trained model to predict the testing cases and print out the predictions and their MSE:

```

>>> predictions = model.predict(X_test)[: , 0]
>>> print(predictions)
[18.078342 17.279167 19.802671 17.54534 16.193192 24.769335
22.12822 30.43017 26.262056 20.982824]

>>> print(np.mean((y_test - predictions) ** 2))
15.72498178190508

```

As you can see, we add layer by layer to the neural network model in the TensorFlow Keras API. We start from the first hidden layer (with 20 nodes), then the second hidden layer (with eight nodes), and finally the output layer (with one unit, the target variable). It is quite similar to building LEGOs. Next, we will look at how to choose the right activation functions.

4) Picking the right activation functions

So far, we have used the ReLU and sigmoid activation functions in our implementations. You may wonder how to pick the right activation function for your neural networks. Detailed answers to when to choose a particular activation function are given next:

Linear: $f(z) = z$. You can interpret this as no activation function. We usually use it in the output layer in regression networks as we don't need any transformation to the outputs.

Sigmoid (logistic) transforms the output of a layer to a range between 0 and 1. You can interpret it as the probability of an output prediction. Therefore, we usually use it in the output layer in binary classification networks. Besides that, we sometimes use it in hidden layers. However, it should be noted that the sigmoid function is monotonic but its derivative is not. Hence, the neural network may get stuck at a suboptimal solution.

Softmax. Softmax is a generalized logistic function used for multiclass classification. Hence, we use it in the output layer in multiclass classification networks.

Tanh is a better version of the sigmoid function with stronger gradients. As you can see in the plots, the derivatives in the tanh function are steeper than those for the sigmoid function. It has a range of -1 to 1. It is common to use the tanh function in hidden layers.

ReLU is probably the most frequently used activation function nowadays. It is the "default" one in hidden layers in feedforward networks. Its range is from 0 to infinity, and both the function itself and its derivative are monotonic. One drawback of the ReLU function is the inability to appropriately map the negative part of the input where all negative inputs are transformed to zero. To fix the "dying negative" problem in ReLU, Leaky ReLU was invented to introduce a small slope in the negative part. When $z < 0$, $f(z) = az$, where a is usually a small value, such as 0.01. To recap, ReLU is usually in hidden layer activation. You can try Leaky ReLU if ReLU doesn't work well. Sigmoid and tanh can be used in hidden layers but are not recommended in deep networks with many layers. For the output layer, linear activation (or no activation) is used in the regression network; sigmoid is for the binary classification network and softmax is for the multiple classification case.

Picking the right activation is important, and so is avoiding overfitting in neural networks. Let's see how to do this in the next part.

5) Preventing overfitting in neural networks

A neural network is powerful as it can derive hierarchical features from data with the right architecture (the right number of hidden layers and hidden nodes). It offers a great deal of flexibility and can fit a complex dataset. However, this advantage will become a weakness if the network is not given enough control over the learning process. Specifically, it may lead to overfitting if a network is only good at fitting to the training set but is not able to generalize to unseen data. Hence, preventing overfitting is essential to the success of a neural network model.

There are mainly three ways to impose restrictions on our neural networks: L1/L2 regularization, dropout, and early stopping. We will discuss dropout and early stopping in the next section.

Dropout

Dropout means ignoring a certain set of hidden nodes during the learning phase of a neural network. And those hidden nodes are chosen randomly given a specified probability. In the forward pass during a training iteration, the randomly selected nodes are temporarily not used in calculating the loss; in the backward pass, the randomly selected nodes are not updated temporarily.

In the following diagram, we choose three nodes in the network to ignore during training:

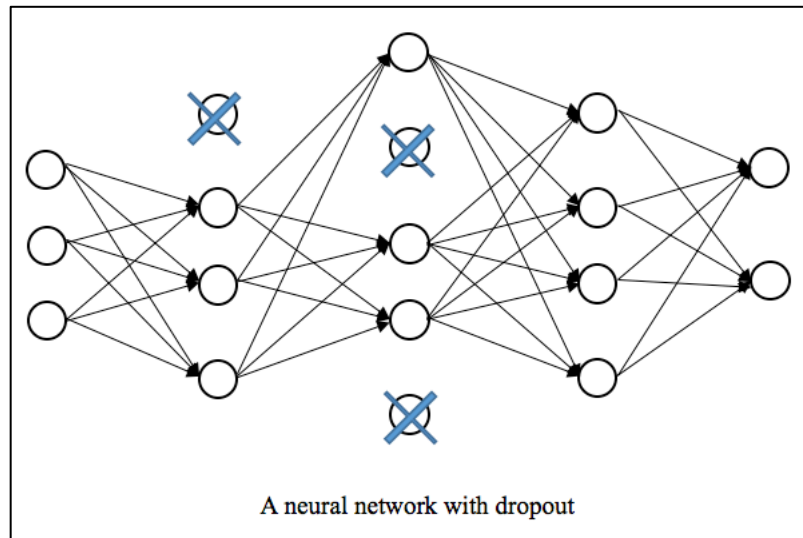


Figure 1: Three nodes to ignore.

Recall that a regular layer has nodes fully connected to nodes from the previous layer and the following layer. It will lead to overfitting if a large network develops and memorizes the co-dependency between individual pairs of nodes. Dropout breaks this co-dependency by temporarily deactivating certain nodes in each iteration. Therefore, it effectively reduces overfitting and won't disrupt learning at the same time.

The fraction of nodes being randomly chosen in each iteration is also called the dropout rate. In practice, we usually set a dropout rate no greater than 50%. In TensorFlow, we use the `tf.keras.layers.Dropout` module to add dropout to a layer.

An example is as follows:

```
>>> model = keras.Sequential([
...     keras.layers.Dense(units=32, activation='relu'),
...     tf.keras.layers.Dropout(0.5)
...     keras.layers.Dense(units=1)
```

In the preceding example, 50% of nodes randomly picked from the 16-node layer are ignored in an iteration during training.

Note: Keep in mind that dropout only occurs in the training phase. In the prediction phase, all nodes are fully connected again.

Early stopping

As the name implies, training a network with early stopping will end if the model performance doesn't improve for a certain number of iterations. The model performance is measured on a validation set that is different from the training set, in order to assess how well it generalizes. During training, if the performance degrades after several (let's say 50) iterations, it means the model is overfitting and not able to generalize well anymore. Hence, stopping the learning early in this case helps prevent overfitting.

In TensorFlow, we use the `tf.keras.callbacks.EarlyStopping` module to incorporate early stopping. I will demonstrate how to use it later in this section.

Now that you've learned about neural networks and their implementation, let's utilize them to solve our stock price prediction problem.

6) Predicting stock prices with neural networks

We will build the stock predictor with TensorFlow in this section. We will start with feature generation and data preparation, followed by network building and training. After that, we will fine-tune the network and incorporate early stopping to boost the stock predictor.

Acquiring data and generating features

For easier reference, we will implement the code for generating features here rather than in later time. We will start by obtaining the dataset we need for our project. Throughout the project, we will acquire stock index price and performance data from Yahoo Finance. For example, on the Historical Data page, <https://finance.yahoo.com/quote/%5EDJI/history?p=%5EDJI>

We can change the Time Period to Dec 01,2005 – Dec10,2005, select Historical Prices in Show, and Daily in Frequency or open this link directly:

<https://finance.yahoo.com/quote/%5EDJI/history?period1=1133395200&period2=1134172800&interval=1d&filter=histoy&frequency=1d&includeAdjustedClose=true>

Then click on the Apply button. Click the Download data button to download the data and name the file 20051201_20051210.csv. You also can download the dataset from [this link](#), but ensure to name it correctly.

We can load the data we just downloaded as follows:

```
>>> mydata = pd.read_csv('20051201_20051210.csv', index_col='Date')
>>> mydata
```

	Open	High	Low	Close
Date				
2005-12-01	10806.030273	10934.900391	10806.030273	10912.570312
2005-12-02	10912.009766	10921.370117	10861.660156	10877.509766
2005-12-05	10876.950195	10876.950195	10810.669922	10835.009766
2005-12-06	10835.410156	10936.200195	10835.410156	10856.860352
2005-12-07	10856.860352	10868.059570	10764.009766	10810.910156
2005-12-08	10808.429688	10847.250000	10729.669922	10755.120117
2005-12-09	10751.759766	10805.950195	10729.910156	10778.580078

	Volume	Adjusted Close
Date		
2005-12-01	256980000.0	10912.570312
2005-12-02	214900000.0	10877.509766
2005-12-05	237340000.0	10835.009766
2005-12-06	264630000.0	10856.860352
2005-12-07	243490000.0	10810.910156
2005-12-08	253290000.0	10755.120117
2005-12-09	238930000.0	10778.580078

Note the output is a pandas DataFrame object. The Date column is the index column, and the rest of the columns are the corresponding financial variables. In the following lines of code, you will see how powerful pandas is at simplifying data analysis and transformation on relational (or table-like) data.

First, we implement feature generation by starting with a sub-function that directly creates features from the original six features, as follows:

```
>>> def add_original_feature(df, df_new):
...     df_new['open'] = df['Open']
...     df_new['open_1'] = df['Open'].shift(1)
...     df_new['close_1'] = df['Close'].shift(1)
...     df_new['high_1'] = df['High'].shift(1)
...     df_new['low_1'] = df['Low'].shift(1)
...     df_new['volume_1'] = df['Volume'].shift(1)
```

Then we develop a sub-function that generates six features related to average close prices:

```
>>> def add_avg_price(df, df_new):
...     df_new['avg_price_5'] =
...         df['Close'].rolling(5).mean().shift(1)
...     df_new['avg_price_30'] =
...         df['Close'].rolling(21).mean().shift(1)
...     df_new['avg_price_365'] =
...         df['Close'].rolling(252).mean().shift(1)
...     df_new['ratio_avg_price_5_30'] =
...         df_new['avg_price_5'] / df_new['avg_price_30']
...     df_new['ratio_avg_price_5_365'] =
...         df_new['avg_price_5'] / df_new['avg_price_365']
...     df_new['ratio_avg_price_30_365'] =
...         df_new['avg_price_30'] / df_new['avg_price_365']
```

Similarly, a sub-function that generates six features related to average volumes is as follows:

```
>>> def add_avg_volume(df, df_new):
...     df_new['avg_volume_5'] =
...         df['Volume'].rolling(5).mean().shift(1)
...     df_new['avg_volume_30'] =
...         df['Volume'].rolling(21).mean().shift(1)
...     df_new['avg_volume_365'] =
...         df['Volume'].rolling(252).mean().shift(1)
...     df_new['ratio_avg_volume_5_30'] =
...         df_new['avg_volume_5'] / df_new['avg_volume_30']
...     df_new['ratio_avg_volume_5_365'] =
...         df_new['avg_volume_5'] / df_new['avg_volume_365']
...     df_new['ratio_avg_volume_30_365'] =
...         df_new['avg_volume_30'] / df_new['avg_volume_365']
```

As for the standard deviation, we develop the following sub-function for the price related features:

```
>>> def add_std_price(df, df_new):
...     df_new['std_price_5'] =
...         df['Close'].rolling(5).std().shift(1)
...     df_new['std_price_30'] =
...         df['Close'].rolling(21).std().shift(1)
...     df_new['std_price_365'] =
...         df['Close'].rolling(252).std().shift(1)
...     df_new['ratio_std_price_5_30'] =
...         df_new['std_price_5'] / df_new['std_price_30']
...     df_new['ratio_std_price_5_365'] =
...         df_new['std_price_5'] / df_new['std_price_365']
...     df_new['ratio_std_price_30_365'] =
...         df_new['std_price_30'] / df_new['std_price_365']
```

Similarly, a sub-function that generates six volume-based standard deviation features is as follows:

```
>>> def add_std_volume(df, df_new):
...     df_new['std_volume_5'] =
...         df['Volume'].rolling(5).std().shift(1)
...     df_new['std_volume_30'] =
...         df['Volume'].rolling(21).std().shift(1)
...     df_new['std_volume_365'] =
...         df['Volume'].rolling(252).std().shift(1)
...     df_new['ratio_std_volume_5_30'] =
...         df_new['std_volume_5'] / df_new['std_volume_30']
...     df_new['ratio_std_volume_5_365'] =
...         df_new['std_volume_5'] / df_new['std_volume_365']
...     df_new['ratio_std_volume_30_365'] =
...         df_new['std_volume_30'] / df_new['std_volume_365']
```

Seven return-based features are generated using the following sub-function:

```
>>> def add_return_feature(df, df_new):
...     df_new['return_1'] = ((df['Close'] - df['Close'].shift(1))
...                           / df['Close'].shift(1)).shift(1)
...     df_new['return_5'] = ((df['Close'] - df['Close'].shift(5))
...                           / df['Close'].shift(5)).shift(1)
...     df_new['return_30'] = ((df['Close'] -
...                             df['Close'].shift(21)) / df['Close'].shift(21)).shift(1)
...     df_new['return_365'] = ((df['Close'] -
...                               df['Close'].shift(252)) / df['Close'].shift(252)).shift(1)
...     df_new['moving_avg_5'] =
...         df_new['return_1'].rolling(5).mean().shift(1)
...     df_new['moving_avg_30'] =
...         df_new['return_1'].rolling(21).mean().shift(1)
...     df_new['moving_avg_365'] =
...         df_new['return_1'].rolling(252).mean().shift(1)
```

Finally, we put together the main feature generation function that calls all the preceding sub-functions:

```
>>> def generate_features(df):
...     """
...     Generate features for a stock/index based on historical
...     price and performance
...     @param df: dataframe with columns "Open", "Close", "High",
...             "Low", "Volume", "Adjusted Close"
...     @return: dataframe, data set with new features
...     """
...     df_new = pd.DataFrame()
...     # 6 original features
...     add_original_feature(df, df_new)
...     # 31 generated features
...     add_avg_price(df, df_new)
...     add_avg_volume(df, df_new)
...     add_std_price(df, df_new)
...     add_std_volume(df, df_new)
...     add_return_feature(df, df_new)
...     # the target
...     df_new['close'] = df['Close']
...     df_new = df_new.dropna(axis=0)
...     return df_new
```

Note that the window sizes here are 5, 21, and 252, instead of 7, 30, and 365 representing the weekly, monthly, and yearly window. This is because there are 252 (rounded) trading days in a year, 21 trading days in a month, and 5 in a week. We can apply this feature engineering strategy on the DJIA data queried from 1988 to 2019 as follows or directly download it from this page:

<https://finance.yahoo.com/quote/%5EDJI/history?period1=567993600&period2=1577750400&interval=1d&filter=history&frequency=1d>

```
>>> data_raw = pd.read_csv('19880101_20191231.csv', index_col='Date')
>>> data = generate_features(data_raw)
```

Take a look at what the data with the new features looks like:

```
>>> print(data.round(decimals=3).head(5))
```

The preceding command line generates the following output:

	open	open_1	close_1	high_1	low_1	volume_1	avg_price_5	...	return_5
return_30	return_365	moving_avg_5	moving_avg_30	moving_avg_365	close				
Date									
1989-01-04	2153.75	2163.21	2144.64	2168.39	2127.14	17310000.0	2165.000	...	-0.011
0.020	0.056		0.001		0.001	0.000	2177.68		
1989-01-05	2184.29	2153.75	2177.68	2183.39	2146.61	15710000.0	2168.000	...	0.007
0.041	0.069	-0.002		0.001		0.000	2190.54		
1989-01-06	2195.89	2184.29	2190.54	2205.18	2173.04	20310000.0	2172.822	...	0.011
0.031	0.068		0.001		0.002	0.000	2194.29		
1989-01-09	2194.82	2195.89	2194.29	2213.75	2182.32	16500000.0	2175.144	...	0.005
0.021	0.148		0.002		0.001	0.000	2199.46		
1989-01-10	2205.36	2194.82	2199.46	2209.11	2185.00	18420000.0	2181.322	...	0.014
0.021	0.131		0.001		0.001	0.001	2193.21		

[5 rows x 38 columns]

Figure 2: dropout, and early stopping.

Since all features and driving factors are ready, we will now focus on regression algorithms that estimate the continuous target variables based on these predictive features.

Training a simple neural network

We prepare data and train a simple neural work with the following steps:

1. We load the stock data, generate features, and label the generate_features function we developed in Chapter 7, Predicting Stock Prices with Regression Algorithms:

```
>>> data_raw = pd.read_csv('19880101_20191231.csv', index_col='Date')
>>> data = generate_features(data_raw)
```

2. We construct the training set using data from 1988 to 2018 and the testing set using data from 2019:

```
>>> start_train = '1988-01-01'
>>> end_train = '2018-12-31'
>>> start_test = '2019-01-01'
>>> end_test = '2019-12-31'
>>> data_train = data.loc[start_train:end_train]
>>> X_train = data_train.drop('close', axis=1).values
>>> y_train = data_train['close'].values
>>> data_test = data.loc[start_test:end_test]
>>> X_test = data_test.drop('close', axis=1).values
>>> y_test = data_test['close'].values
```

3. We need to normalize features into the same or a comparable scale. We do so by removing the mean and rescaling to unit variance:

```
>>> from sklearn.preprocessing import StandardScaler
>>> scaler = StandardScaler()
```

We rescale both sets with the scaler taught by the training set:

```
>>> X_scaled_train = scaler.fit_transform(X_train)
>>> X_scaled_test = scaler.transform(X_test)
```

4. We now build a neural network model using the Keras Sequential API:

```
>>> from tensorflow.keras import Sequential
>>> from tensorflow.keras.layers import Dense
>>> model = Sequential([
...     Dense(units=32, activation='relu'),
...     Dense(units=1)
... ])
```

The network we begin with has one hidden layer with 32 nodes followed by a ReLU function.

5. And we compile the model by using Adam as the optimizer with a learning rate of 0.1 and MSE as the learning goal:

```
>>> model.compile(loss='mean_squared_error',
...               optimizer=tf.keras.optimizers.Adam(0.1))
```

6. After defining the model, we now train it against the training set:

```
>>> model.fit(X_scaled_train, y_train, epochs=100, verbose=True)
Train on 7558 samples
Epoch 1/100
7558/7558 [=====] - 1s 175us/sample -
loss: 31078305.1905
Epoch 2/100
7558/7558 [=====] - 0s 58us/sample -
loss: 2062612.2298
Epoch 3/100
7558/7558 [=====] - 0s 56us/sample -
loss: 474157.7456
.....
Epoch 98/100
7558/7558 [=====] - 0s 56us/sample -
loss: 21777.9346
Epoch 99/100
7558/7558 [=====] - 0s 55us/sample -
loss: 19343.1628
Epoch 100/100
7558/7558 [=====] - 0s 52us/sample -
loss: 20780.1686
```

7. Finally, we use the trained model to predict the testing data and display metrics:

```
>>> from sklearn.metrics import mean_squared_error, mean_
absolute_error, r2_score
>>> print(f'MSE: {mean_squared_error(y_test, predictions):.3f}')
MSE: 43212.312
>>> print(f'MAE: {mean_absolute_error(y_test,
predictions):.3f}')
MAE: 160.936
>>> print(f'R^2: {r2_score(y_test, predictions):.3f}')
R^2: 0.962
```

We achieve 0.962 R2 with a simple neural network model!

Fine-tuning the neural network

Can we do better? Of course, we haven't fine-tuned the hyperparameters yet. We perform model fine-tuning in TensorFlow with the following steps:

1. We rely on the hparams module in TensorFlow, so we import it first:

```
>>> from tensorboard.plugins.hparams import api as hp
```

2. We want to tweak the number of hidden nodes in the hidden layer (again, we are using one hidden layer for this example), the number of training iterations, and the learning rate. We pick the following values of hyperparameters to experiment on:

```
>>> HP_HIDDEN = hp.HParam('hidden_size', hp.Discrete([64, 32,
16]))
>>> HP_EPOCHS = hp.HParam('epochs', hp.Discrete([300, 1000]))
>>> HP_LEARNING_RATE = hp.HParam('learning_rate',
hp.RealInterval(0.01, 0.4))
```

Here, we experiment with three options for the number of hidden nodes (discrete value), 16, 32, and 64; we use two options for the number of iterations (discrete value), 300 and 1000; and we use the range of 0.01 to 4 for the learning rate (continuous value).

3. After initializing the hyperparameters to optimize, we now create a function to train and validate the model that will take the hyperparameters as arguments, and output the performance:

```

>>> def train_test_model(hparams, logdir):
...     model = Sequential([
...         Dense(units=hparams[HP_HIDDEN],
...               activation='relu'),
...         Dense(units=1)
...     ])
...     model.compile(loss='mean_squared_error',
...                   optimizer=tf.keras.optimizers.Adam(
...                       hparams[HP_LEARNING_RATE]),
...                   metrics=['mean_squared_error'])
...     model.fit(X_scaled_train, y_train,
...               validation_data=(X_scaled_test, y_test),
...               epochs=hparams[HP_EPOCHS], verbose=False,
...               callbacks=[
...                 tf.keras.callbacks.TensorBoard(logdir),
...                 hp.KerasCallback(logdir, hparams),
...                 tf.keras.callbacks.EarlyStopping(
...                     monitor='val_loss', min_delta=0,
...                     patience=200, verbose=0,
...                     mode='auto',
...                 )
...             ],
...             )
...     _, mse = model.evaluate(X_scaled_test, y_test)
...     pred = model.predict(X_scaled_test)
...     r2 = r2_score(y_test, pred)
...     return mse, r2

```

Here, we build, compile, and fit a neural network model based on the given hyperparameters, including the number of hidden nodes, the learning rate, and the number of training iterations. There's nothing much different here from what we did before. But when we train the model, we also run several callback functions, including updating TensorBoard using `tf.keras.callbacks.TensorBoard(logdir)`, logging hyperparameters and metrics using `hp.KerasCallback(logdir, hparams)`, and early stopping using `tf.keras.callbacks.EarlyStopping(...)`.

The TensorBoard callback function is straightforward. It provides visualization for the model graph and metrics during training and validation. The hyperparameters logging callback logs the hyperparameters and metrics. The early stopping callback monitors the performance on the validation set, which is the testing set in our case. If the MSE doesn't decrease after 200 epochs, it stops the training process. At the end of this function, we output the MSE and R2 of the prediction on the testing set.

4. Next, we develop a function to initiate a training process with a combination of hyperparameters to be assessed and to write a summary with the metrics for MSE and R2 returned by the `train_test_model` function:

```

>>> def run(hparams, logdir):
...     with tf.summary.create_file_writer(logdir).as_default():
...         hp.hparams_config(
...             hparams=[HP_HIDDEN, HP_EPOCHS,
...                       HP_LEARNING_RATE],
...             metrics=[hp.Metric('mean_squared_error',
...                               display_name='mse'),
...                     hp.Metric('r2', display_name='r2')],
...         )
...         mse, r2 = train_test_model(hparams, logdir)
...         tf.summary.scalar('mean_squared_error', mse,
...                             step=1)
...         tf.summary.scalar('r2', r2, step=1)

```

5. We now train the model for each different combination of the hyperparameters in a gridsearch manner:

```

>>> for hidden in HP_HIDDEN.domain.values:
...     for epochs in HP_EPOCHS.domain.values:
...         for learning_rate in
...             tf.linspace(HP_LEARNING_RATE.domain.min_value,
...                           HP_LEARNING_RATE.domain.max_value,
...                           5):
...             hparams = {
...                 HP_HIDDEN: hidden,
...                 HP_EPOCHS: epochs,
...                 HP_LEARNING_RATE:
...                     float("%.2f"%float(learning_rate)),
...             }
...             run_name = "run-%d" % session_num
...             print('--- Starting trial: %s' % run_name)
...             print({h.name: hparams[h] for h in hparams})
...             run(hparams, 'logs/hparam_tuning/' + run_name)
...             session_num += 1

```

For each experiment, a discrete value (the number of hidden nodes and iterations) is picked from the predefined value pool and a continuous value (the learning rate) is chosen from five evenly spaced values over the interval (from 0.01 to 0.4). It will take a few minutes to run these experiments. You will see the following output:

```

--- Starting trial: run-0
{'hidden_size': 16, 'epochs': 300, 'learning_rate': 0.01}
2020-04-29 08:06:43.149021: I tensorflow/core/profiler/lib/
profiler_session.cc:184] Profiler session started.
.....
=====] - 0s 42us/
sample - loss: 62625.1632 - mean_squared_error: 55865.6680
.....
.....
.....
--- Starting trial: run-29
{'hidden_size': 64, 'epochs': 1000, 'learning_rate': 0.4}
2020-04-29 08:28:03.036671: I tensorflow/core/profiler/lib/
profiler_session.cc:184] Profiler session started.
.....
=====] - 0s 54us/
sample - loss: 51182.3352 - mean_squared_error: 59099.1250

```

6. You will notice that a new folder, logs, is created after the experiments start. It contains the training and validation performance for each experiment. After 30 experiments finish, it's time to launch TensorBoard. We use the following command:

```

tensorboard --logdir ls/hparam_tuning
Serving TensorBoard on localhost; to expose to the network, use
a proxy or pass --bind_all
TensorBoard 2.0.0 at http://localhost:6006/ (Press CTRL+C to
quit)

```

Once it is launched, you will see the beautiful dashboard at <http://localhost:6006/>. See the screenshot of the expected result here:

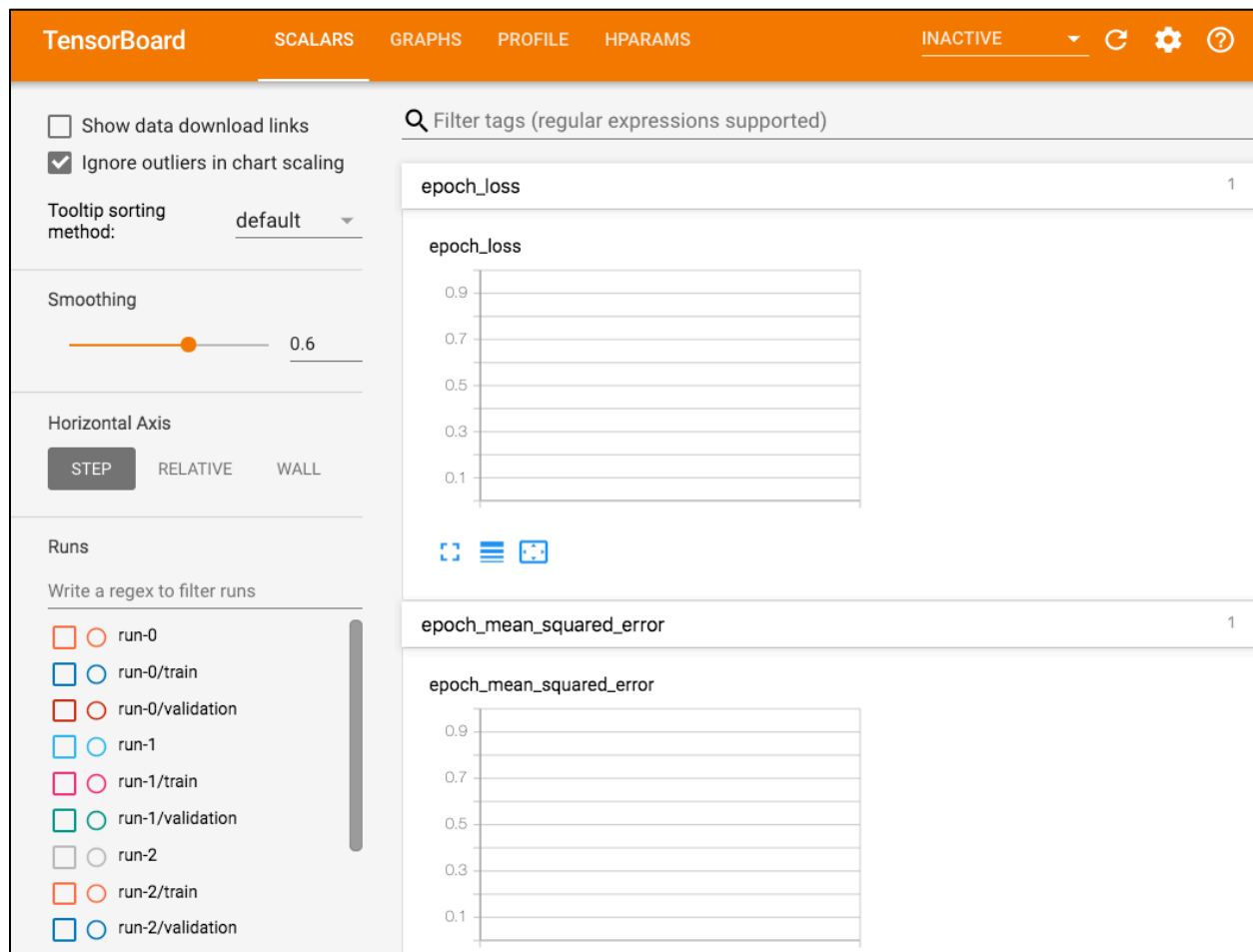


Figure 3: Screenshot of TensorBoard.

Click on the HPARAMS tab to see the hyperparameter logs. You can see all the hyperparameter combinations and the respective metrics (MSE and R2) displayed in a table, as shown here:

TensorBoard						
SCALARS GRAPHS PROFILE HPARAMS						
INACTIVE						
Hyperparameters	TABLE VIEW		PARALLEL COORDINATES VIEW		SCATTER PLOT MATRIX VIEW	
	Trial ID	Show Metrics	hidden_size	epochs	learning_rate	r2
☒ hidden_size	040324d52ab617...	☐	64.000	300.00	0.21000	0.96836
☒ 16.000	0979d01d62d519...	☐	16.000	300.00	0.40000	0.96778
☒ 32.000	1a60de513b2319...	☐	32.000	300.00	0.21000	0.96969
☒ 64.000	485f4ee17ecb588...	☐	64.000	1000.0	0.11000	0.91668
☒ epochs	6552154cd58ee14...	☐	32.000	1000.0	0.21000	0.96489
☒ 300.00	6cb44a3856f106e...	☐	32.000	1000.0	0.30000	0.94610
☒ 1000.0	7040706da3e91e...	☐	16.000	300.00	0.30000	0.96650
☒ learning_rate	75440855f618f11...	☐	16.000	1000.0	0.21000	0.97122
Min	795049416b5fa81...	☐	64.000	1000.0	0.40000	0.94846
-infinity	7b5f9905ea11db8...	☐	16.000	1000.0	0.010000	0.96971
Max	80a1dbb4fb82af9...	☐	64.000	1000.0	0.30000	0.94267
+infinity	82fecc9de850858...	☐	32.000	300.00	0.40000	0.96880
	8791a9be31c112...	☐	16.000	1000.0	0.40000	0.94872
Metrics	8b4385335083a2f...	☐	16.000	300.00	0.11000	0.96044
☒ mse	8d27faae826dcd9...	☐	64.000	300.00	0.40000	0.96749
Min	8f89d44a41663e3...	☐	16.000	1000.0	0.11000	0.92637
-infini	982dce79d07f59d...	☐	32.000	1000.0	0.11000	0.94175
Max	a13f6d949f5727b...	☐	64.000	300.00	0.30000	0.95169
+infini	a5b9a7e44d9012...	☐	32.000	1000.0	0.010000	0.96680
	bf45446cad23c5e...	☐	16.000	1000.0	0.30000	0.96299
☒ r2	d28a4bbb534c83f...	☐	16.000	300.00	0.21000	0.96780
Min	d4a2a530db443d...	☐	16.000	300.00	0.010000	0.95128
-infini	d8b1aea9727190...	☐	32.000	300.00	0.30000	0.97005
Max	df1fd0e02334141...	☐	64.000	1000.0	0.010000	0.96055
+infini						
Status						
☒ Unknown						
☒ Success						
☒ Failure						
☒ Running						
Sorting						
Sort by						
Direction						

Figure 4: Screenshot of TensorBoard for hyperparameter tuning.

The combination of (hidden_size=16, epochs=1000, learning_rate=0.21) is the best performing one, with which we achieve an R2 of 0.97122.

- Finally, we use the optimal model to make predictions:

```
>>> model = Sequential([
...     Dense(units=16, activation='relu'),
...     Dense(units=1)
... ])
>>> model.compile(loss='mean_squared_error',
...                 optimizer=tf.keras.optimizers.Adam(0.21))
>>> model.fit(X_scaled_train, y_train, epochs=1000,
...             verbose=False)
>>> predictions = model.predict(X_scaled_test)[: , 0]
```

- Plot the prediction along with the ground truth as follows:

```

>>> import matplotlib.pyplot as plt
>>> plt.plot(data_test.index, y_test, c='k')
>>> plt.plot(data_test.index, predictions, c='b')
>>> plt.plot(data_test.index, predictions, c='r')
>>> plt.plot(data_test.index, predictions, c='g')
>>> plt.xticks(range(0, 252, 10), rotation=60)
>>> plt.xlabel('Date')
>>> plt.ylabel('Close price')
>>> plt.legend(['Truth', 'Neural network prediction'])
>>> plt.show()

```

Refer to the following screenshot for the end result:

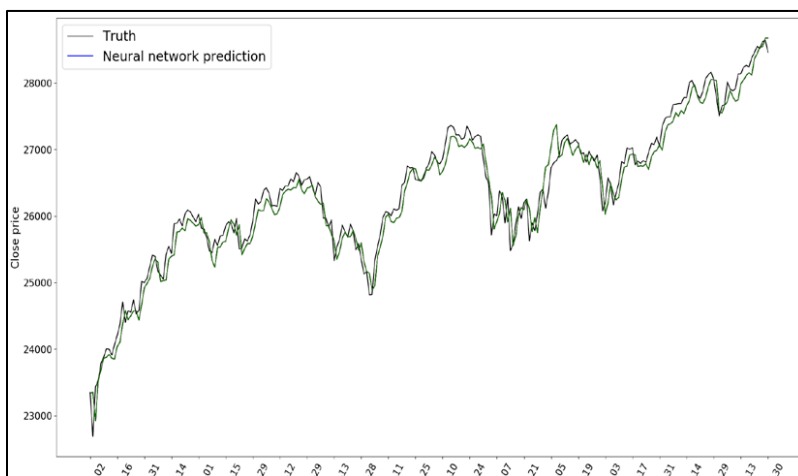


Figure 5: Prediction and ground truth of stock prices.

The fine-tuned neural network does a good job of predicting stock prices!

In this assignment, we further improved the neural network stock predictor by utilizing the hparams module in TensorFlow. Feel free to use more hidden layers and re-run model fine-tuning to see whether you can get a better result.

Summary

In this assignment, we worked on the stock prediction project again, but with neural networks this time. We started with a detailed explanation of neural networks, including the essential components (layers, activations, feedforward, and backpropagation), and transitioned to DL. We moved on to implementations from scratch with scikit-learn and TensorFlow. You also learned about ways to avoid overfitting, such as dropout and early stopping. Finally, we applied what we covered previously to solve our stock price prediction problem.